

1.3 开发环境搭建

所属阶段： 第一阶段 · ROS 2 入门与环境搭建

讲次数量： 8 节

预计时长： ~55 分钟

前置依赖： 1.2 ROS 2 架构概览

学习目标

完成本节后，学员将能够：

- 在 VirtualBox 虚拟机或 WSL2 中搭建 Ubuntu 24.04 开发环境
- 在 Ubuntu 24.04 上完成 ROS 2 Jazzy 的全流程安装与验证
- 配置 VS Code 及 ROS 2 相关插件，建立高效的开发 workflow
- 理解 colcon 构建系统的工作原理
- 创建、构建并激活第一个 ROS 2 工作空间
- 独立排查常见的环境配置错误

讲次 1：选择并准备你的开发环境

知识点

本课程推荐使用 **Ubuntu 24.04 LTS + ROS 2 Jazzy**。请根据你的操作系统选择**一条**安装路径，并按对应讲次操作即可，无需重复执行：

方案	适用人群	后续路径
WSL2	Windows 10/11 用户	按 讲次 2 → 讲次 3 → 讲次 4 完成 WSL 准备、ROS 2 安装和验证
原生 Ubuntu / 双系统	追求最佳仿真性能	安装 Ubuntu 24.04 LTS，继续 讲次 3 → 讲次 4
VirtualBox 虚拟机	Windows/macOS 初学者	8 GB 内存、80 GB 磁盘、启用 3D 加速，继续 讲次 3 → 讲次 4

分流提示： 只有操作系统准备步骤不同。Windows 用户先做讲次 2；所有人都在讲次 3 执行同一份 ROS 2 安装命令，并在讲次 4 使用同一套验收标准。

VirtualBox 准备： 下载 [VirtualBox](#) 和 [Ubuntu 24.04 ISO](#) → 创建虚拟机（8192 MB 内存、80 GB 磁盘、启用 3D 加速）→ 安装系统后再安装 Guest Additions，然后前往讲次 3。

讲次 2：准备 WSL2 Ubuntu (Windows 用户专属)

知识点

WSL2 (Windows Subsystem for Linux 2) 让 Windows 10/11 运行 Linux 环境。本讲次只处理 Windows 专属事项：安装 Ubuntu 24.04、分配资源和验证图形界面；ROS 2 的公共安装流程统一放在讲次 3。

前置条件

项目	要求
操作系统	Windows 10 Build 19044+ 或 Windows 11 (GUI 应用要求)
WSL 版本	WSL2 (默认)
内存	建议 8 GB 以上
磁盘空间	建议预留 80 GB 以上

步骤 1：安装 WSL2 (支持 C 盘或 D 盘)

适用系统： Windows 10 Build 19044+ 或 Windows 11。较旧系统请先完成 Windows Update；课程需要的 Linux GUI 支持以微软当前 WSL 文档为准。

提示： 以下命令在 **Windows PowerShell (管理员)** 中执行，不是在 Ubuntu 里。WSL 让你在 Windows 里运行一个完整的 Linux 系统，ROS 2 就装在这个 Linux 里。

```
# ===== 第 1 步：启用 WSL2（以管理员身份运行 PowerShell） =====

# wsl --install --no-distribution
# 只安装 WSL 核心组件，暂不下载 Linux 发行版（如 Ubuntu）
# 相当于先搭好“运行 Linux 的框架”，后面再选择装哪个 Linux
wsl --install --no-distribution

# 如果提示重启，先重启 Windows，再回到管理员 PowerShell 执行：
wsl --update
wsl --version

# wsl --set-default-version 2
# 设置后续新发行版默认使用 WSL2（本课程的 Linux 仿真路径基于 WSL2）
wsl --set-default-version 2

# ===== 第 2 步：安装 Ubuntu 24.04 =====
# Ubuntu 24.04 是 ROS 2 Jazzy 官方支持的 Linux 版本，二者必须配套

# 方案 A：安装到 C 盘（默认位置，适合 C 盘空间充足的用户）
wsl --install -d Ubuntu-24.04

# 方案 B：安装到 D 盘（需要较新的 Store 版 WSL，二选一，不要与方案 A 都执行）
# --location 指定虚拟磁盘 .vhdx 的存放路径，Linux 系统文件都保存在这个文件里
# wsl --install -d Ubuntu-24.04 --location D:\WSL\Ubuntu-24.04

# 安装成功后，终端会显示类似以下提示（仅为示例输出，不要当作命令执行）：
# 正在下载：Ubuntu 24.04 LTS
# 正在安装：Ubuntu 24.04 LTS
# 已成功安装分发。可以通过“wsl.exe -d Ubuntu-24.04”启动它
# 正在启动 Ubuntu-24.04...

# ----- 命令参数说明 -----
# wsl          → 调用 Windows 的 WSL 管理工具
# --install    → 执行安装操作
# -d Ubuntu-24.04 → -d 是 --distribution 缩写，指定安装 Ubuntu 24.04 这个发行版
# --location D:\WSL\Ubuntu-24.04 → 把 Linux 虚拟磁盘装到 D 盘指定文件夹，而非默认 C 盘
```

只执行方案 A 或方案 B，不要两条都执行。 如果 `--location` 报“不识别的参数”，先确认 `wsl --update` 已成功；仍不支持时可先按方案 A 安装，再参考微软 WSL 导入/导出文档迁移。可用 `wsl -l -v` 确认发行版名称和 WSL 版本。

根据提示创建 Linux 用户名和密码。密码输入时不会回显，这是正常现象。该密码用于 `sudo` 获取管理员权限，请使用自己能记住但不容易被猜到的密码，不要使用 `1`、`123456` 或与用户名相同的弱密码。

```
Provisioning the new WSL instance Ubuntu-24.04
This might take a while...
Create a default Unix user account: lqf      # 输入你的 Linux 用户名（登录 WSL 时使用）
New password:                               # 输入密码（输入时屏幕不显示，正常现象）
Retype new password:                         # 再次输入密码确认
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

关于 sudo: 后面很多命令以 `sudo` 开头, 表示"以管理员身份执行"。系统会要求输入刚才设置的密码。这是 Linux 的安全机制, 不是报错。

此时命令行的 Ubuntu 就安装完毕了。

步骤 2: 配置 WSL2 资源限制

WSL2 会动态使用主机资源。只有在编译或仿真影响 Windows 使用时, 才需要通过 `.wslconfig` 设置上限; 8 GB 是建议值, 不应超过物理内存可承受范围。

创建或编辑 `C:\Users\<你的用户名>\.wslconfig` (Windows 用户目录下, 用记事本即可), 添加以下内容:

```
[wsl2]
memory=8GB
processors=4
swap=2GB
localhostForwarding=true
guiApplications=true
```

然后重启 WSL 使配置生效:

```
# wsl --shutdown → 关闭所有 WSL 实例 (类似"关机", 不会删除数据)
wsl --shutdown

# wsl → 重新启动 WSL, 新配置在此刻生效
wsl
```

步骤 3: 进入 WSL2 并确认 Ubuntu 版本

提示: 从这里开始, 所有命令都在 **WSL2 的 Ubuntu 终端**里执行, 不是在 Windows PowerShell 里。若尚未进入 Ubuntu, 请先在 PowerShell 执行 `wsl -d Ubuntu-24.04`, 进入后再运行下面的 Bash 命令。

```
# 必须确认 Codename 为 noble (Ubuntu 24.04 的内部代号), 才能安装 ROS 2 Jazzy
lsb_release -a
# 预期输出 Codename: noble
```

系统更新、开发工具和 ROS 2 都在讲次 3 的公共流程中安装, 这里不再维护第二份命令。

步骤 4: 验证 WSL2 GUI 支持 (WSLg)

ROS 2 的 RViz 2、Turtlesim 等工具需要图形界面。Windows 10 Build 19044+ 和 Windows 11 在使用较新的 Store 版 WSL 时都可以使用 WSLg。先在 PowerShell 执行 `wsl --update` 和 `wsl --shutdown`, 重新进入 Ubuntu 后再验证, 不要先手工覆盖 `DISPLAY`。

```
# WSLg 正常时通常能看到 WAYLAND_DISPLAY; 为空不代表一定失败, 仍以实际窗口为准
echo "$WAYLAND_DISPLAY"

# x11-apps 包含 xclock, 用来实际验证 GUI
sudo apt install -y x11-apps

# 运行 xclock → 如果弹出一个时钟窗口, 说明 GUI 支持正常
# 后续 Turtlesim、RViz 2 等图形程序才能正常显示
xclock
```

如果看到时钟窗口弹出, 说明 GUI 支持正常。

仅当更新 WSL 后仍没有 WSLg 时, 才考虑 VcXsrv 等第三方 X Server:

1. 从项目官方发布页下载安装 VcXsrv: <https://sourceforge.net/projects/vcxsrv/files/latest/download>
2. 启动 XLaunch, 按以下步骤配置:
3. Display settings: 选择 "Multiple windows"
4. Client: 选择 "Start no client"
5. Extra settings: 为排障可临时使用 "Disable access control", 但它会降低访问控制强度; 只允许 Windows 防火墙的 "专用网络", 测试完成后关闭 X Server
6. Configuration complete: 点击 "Finish"
7. 在 WSL2 中设置 DISPLAY 环境变量 (告诉 Linux 把图形窗口发送到哪里显示):

```
# 先只在当前终端测试, 确认 xclock 能显示后再决定是否写入 ~/.bashrc
export DISPLAY="$(awk '/nameserver / {print $2; exit}' /etc/resolv.conf):0"
xclock
```

技巧: 在 WSL 中开多个终端

ROS 2 开发经常需要同时运行多个节点, 每个节点占一个终端。以下是几种开多终端的方法:

方法 1: 新开一个 PowerShell/CMD 窗口 (最常用)

```
# 在 Windows 再开一个 PowerShell 或 CMD 窗口, 输入 wsl 即可进入 Ubuntu
# 每个窗口是独立的终端, 互不影响 (适合同时跑 turtlesim 和键盘控制)
wsl
```

方法 2: 使用 tmux 分屏 (推荐长期使用)

tmux 是终端复用工具, 在一个窗口里分出多个面板, 类似 IDE 的分屏功能。

```
# 安装 tmux
sudo apt install tmux -y

# 常用操作（先按 Ctrl+B 松开，再按后面的键）：
tmux                # 启动 tmux 会话
Ctrl+B %            # 左右分屏（一个面板跑节点，一个面板看日志）
Ctrl+B "            # 上下分屏
Ctrl+B 方向键       # 在面板之间切换
Ctrl+B d            # 脱离会话（后台保持运行，关闭窗口也不中断节点）
tmux attach          # 重新连接到之前的会话
```

WSL2 常见问题

问题	原因	解决方案
C 盘空间不足	WSL 默认安装在 C 盘	使用导出/导入迁移到 D 盘（见步骤 1）
GUI 应用无法显示	WSL/显卡驱动未更新，或 WSLg 不可用	Windows 10/11 先执行 <code>wsl --update</code> ；确认驱动后再考虑 VcXsrv
网络连接问题	WSL2 网络配置	检查 Windows 防火墙，尝试 <code>wsl --shutdown</code> 后重启
性能问题	内存/CPU 分配不足	调整 <code>.wslconfig</code> 中的 memory 和 processors
构建很慢	在 <code>/mnt/c</code> 、 <code>/mnt/d</code> 等挂载盘上产生大量小文件	源码可按协作需要放挂载盘，但建议把 <code>build/install/log</code> 放到 WSL 的 Linux 文件系统或仓库外目录

完成 `xclock` 验证后，继续讲次 3。后续工作空间建议放在 `~/ros2_ws`；若源码必须与 Windows 共享，至少将高频写入的 `build/install/log` 放到 Linux 文件系统或仓库外目录。

讲次 3：在 Ubuntu 24.04 安装 ROS 2 Jazzy（所有环境共用）

本讲次适用于 WSL2、原生 Ubuntu、双系统和 VirtualBox。Windows 用户应先完成讲次 2，再回到这里；以下 Bash 命令都在 Ubuntu 终端执行。

知识点

步骤 1：系统基础配置

进入 Ubuntu 24.04 后，先更新系统并安装常用工具。`lsb_release` 必须显示 `Codename: noble`，否则不要继续安装 Jazzy：

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl wget vim htop tree net-tools
lsb_release -a # 确认 Codename: noble
```

步骤 2：安装 ROS 2 Jazzy

在 Ubuntu 24.04 中执行以下命令安装 ROS 2 Jazzy：


```

sudo apt update && sudo apt install -y locales software-properties-common curl
sudo locale-gen en_US en_US.UTF-8 && sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

sudo add-apt-repository universe -y
sudo apt update && sudo apt install -y curl
export ROS_APT_SOURCE_VERSION="$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F 'tag_name' | awk -F\" '{print $4}')"
curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb

sudo apt update
sudo apt install -y \
  ros-jazzy-desktop \
  ros-jazzy-turtlesim \
  python3-colcon-common-extensions \
  python3-rosdep \
  python3-argcomplete \
  ros-dev-tools

grep -Fqx 'source /opt/ros/jazzy/setup.bash' ~/.bashrc || \
  echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
grep -Fqx 'eval "$(register-python-argcomplete ros2)"' ~/.bashrc || \
  echo 'eval "$(register-python-argcomplete ros2)"' >> ~/.bashrc
source ~/.bashrc
if [ ! -f /etc/ros/rosdep/sources.list.d/20-default.list ]; then
  sudo rosdep init
fi
rosdep update

```

这套命令依次完成 UTF-8 locale、Universe 仓库、官方 `ros2-apt-source`、桌面版与开发工具安装、环境激活和 rosdep 初始化。命令以 2026-07 的 [ROS 2 Jazzy Ubuntu 安装文档](#) 核对；仓库基础设施更新后应以官方页面为准。

完成标准

- 当前终端执行 `echo "$ROS_DISTRO"` 输出 `jazzy`
 - `/etc/ros/rosdep/sources.list.d/20-default.list` 已存在，且 `rosdep update` 没有被网络或权限错误中断
 - WSL2、原生和虚拟机用户都继续讲次 4，不在此处重复一套验证命令
-

讲次 4: 【LAB】统一验证 ROS 2 Jazzy 安装

本讲次适用于 WSL2、原生 Ubuntu 和虚拟机。所有环境使用相同的命令和通过标准；WSL2 的窗口若不显示，再回到讲次 2 排查 WSLg。

知识点

安装完成后，执行以下验证命令：

```
# 1. 确认系统版本
lsb_release -a
# 预期: Codename: noble

# 2. 检查 ROS 2 发行版
echo $ROS_DISTRO
# 预期: jazzy

# 3. 运行系统诊断: 逐条阅读 WARNING, 不把未知警告直接当作通过
ros2 doctor
# 预期: All checks passed, 或只有已理解并记录原因的警告

# 4. Turtlesim 验证 (需要两个终端)
ros2 run turtlesim turtlesim_node      # 终端 1: 启动海龟仿真
ros2 run turtlesim turtle_teleop_key   # 终端 2: 键盘控制
```

预期: 弹出一个蓝色窗口，中间有一只海龟。



常见错误与解决方案

错误信息	原因	解决方案
<code>ros2: command not found</code>	未 source 环境	<code>source /opt/ros/jazzy/setup.bash</code>
<code>E: Unable to locate package ros-jazzy-desktop</code>	APT 源未添加或系统版本不对	执行 <code>dpkg -s ros2-apt-source</code> 并确认 Ubuntu 24.04; 再按讲次 3 重做源配置
<code>rosdep init</code> 报错已初始化	重复执行	忽略, 直接 <code>rosdep update</code>
Turtlesim 窗口不显示 (WSL2)	WSLg/显卡驱动未就绪	Windows 10/11 先更新 WSL 与显卡驱动; 再按讲次 2 排查
Package 'turtlesim' not found	未安装 turtlesim 包	<code>sudo apt install ros-jazzy-turtlesim</code>

通过标准: `ROS_DISTRO=jazzy`、`ros2 doctor` 无未知错误、Turtlesim 窗口可见且键盘节点能够驱动海龟。只看到安装命令返回成功, 不等于完成验收。

一键诊断脚本： 仓库提供 `stage1_code/scripts/check_env.sh` , 会自动检查 ROS 环境、GUI、常用工具与工作空间：

```
``bash bash /stage1_code/scripts/check_env.sh
```

工作空间不在 `~/ros2_ws` 时可显式传入路径：

```
bash /stage1_code/scripts/check_env.sh /path/to/your_ws ``
```

附录 A: ROS 2 Humble 安装 (Ubuntu 22.04)

本课程以 **ROS 2 Jazzy (Ubuntu 24.04)** 为主。以下内容仅供已有 Ubuntu 22.04 环境或需要对接 Humble 项目的学员参考。

安装命令

Humble 必须运行在其支持的 Ubuntu 22.04 主路径上。先按 [ROS 2 Humble 官方 Ubuntu 安装文档](#) 配置 APT 源，再安装以下包；不要只在 Jazzy 环境里替换包名前缀。

```
sudo apt update
sudo apt install -y ros-humble-desktop ros-humble-turtlesim python3-colcon-common-extensions python3-pip
grep -Fqx 'source /opt/ros/humble/setup.bash' ~/.bashrc || \
    echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
source ~/.bashrc
if [ ! -f /etc/ros/rosdep/sources.list.d/20-default.list ]; then sudo rosdep init; fi
rosdep update
```

验证

```
echo $ROS_DISTRO
# 预期: humble

ros2 doctor
ros2 run turtlesim turtlesim_node
```

讲次 5：配置 VS Code 与 ROS 2 插件

知识点

VS Code 是 ROS 2 开发最流行的 IDE。通过合理的插件配置，可以获得语法高亮、自动补全、调试和 Launch 文件支持。

Ubuntu 上通过 `sudo snap install code --classic` 或官方 APT 源安装 VS Code。Windows 用户使用本地 VS Code + **Remote-WSL** 插件连接 WSL2。

推荐插件

在 VS Code 扩展市场 (Ctrl+Shift+X) 中搜索并安装：

插件名称	发布者	用途
Robot Developer Extensions for ROS 2	Ranch Hand Robotics	ROS 2 辅助命令、Launch 调试；由原 Microsoft <code>ms-iot.vscode-ros</code> (已弃用) 作者维护，不是运行 ROS 2 的必需条件
C/C++ Extension Pack	Microsoft	C++ 智能提示、调试、CMake 支持
Python	Microsoft	Python 智能提示、调试、Linting
Python Debugger	Microsoft	Python 断点调试，随 Python 扩展一起安装 (<code>ms-python.debugpy</code>)
Black Formatter	Microsoft	Python 代码自动格式化 (配合本节的 <code>settings.json</code> 模板)
CMake Tools	Microsoft	CMake 项目配置和构建
WSL	Microsoft	Windows 本地 VS Code 连接 WSL 文件系统和终端

安装命令 (可选，通过 CLI 批量安装)：

```
# code --install-extension <扩展ID> → 通过命令行安装 VS Code 扩展，免去界面里逐个搜索
code --install-extension Ranch-Hand-Robotics.rde-ros-2      # ROS 2 插件：Launch 调试等 (原 ms-iot.vscode-ros)
code --install-extension ms-vscode.cpptools-extension-pack   # C/C++ 智能提示与调试
code --install-extension ms-python.python                   # Python 智能提示与调试 (含 Python Debugger)
code --install-extension ms-python.black-formatter           # Python 自动格式化
code --install-extension ms-vscode.cmake-tools               # CMake 项目配置与构建
code --install-extension ms-vscode-remote.remote-wsl         # Windows 侧连接 WSL
```

Extensions: Marketplace

R	ROS	Installed
	Microsoft - ROS Development Support	
C	C/C++	Installed
	Microsoft - IntelliSense, debugging	
P	Python	Installed
	Microsoft - Linting, debugging	
C	CMake Tools	Installed
	Microsoft - CMake integration	
X	XML	Installed
	Red Hat - XML Language Support	
U	URDF Preview	
	smilerobotics - URDF visualization	

ROS 2 Development Extensions

Recommended extensions for ROS 2 Jazzy:

1. ROS - Launch, build, debug support
2. C/C++ - IntelliSense, code navigation
3. Python - Linting, debugging, formatting
4. CMake Tools - Build system integration
5. XML - URDF/SDF/Launch XML support
6. URDF Preview - 3D model visualization

Install: Ctrl+Shift+X -> Search extension name

VS Code 工作区配置

打开 ROS 2 工作空间时，VS Code 需要正确识别 ROS 2 环境。创建 `.vscode/settings.json`（完整模板见仓库 `stage1_code/vscode/settings.json`，可直接复制）：

```
{
  "ros.distro": "jazzy",
  "python.analysis.extraPaths": [
    "/opt/ros/jazzy/lib/python3.12/site-packages"
  ],
  "cmake.configureOnOpen": false,
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.formatOnSave": true
  }
}
```

[python] 段依赖上文安装的 **Black Formatter** 扩展；如果没有安装该扩展，删除这一段即可，不影响其余配置。

colcon 的 C++ 编译数据库通常位于 `build/<包名>/compile_commands.json`，并不存在通用的 `build/compile_commands.json`。需要 C++ IntelliSense 时，构建时加入 `--cmake-args -DCMAKE_EXPORT_COMPILE_COMMANDS=ON`，再把 VS Code 的 `C_Cpp.default.compileCommands` 指向具体包，或使用专门的 `compile-commands` 汇总工具。

Windows 用户：安装 WSL 插件 → Connect to WSL → `code ~/ros2_ws` → 在 WSL 内安装 ROS 插件。

讲次 6: colcon 构建系统概述

知识点

colcon 是 ROS 2 标准构建系统（取代 ROS 1 的 catkin），统一管理 C++（CMake）和 Python（setuptools）包的编译。

colcon 构建流程



构建后必须执行: `source install/setup.bash`

核心命令：

```
colcon build                # 构建所有包
colcon build --packages-select my_pkg    # 构建指定包
colcon build --event-handlers console_direct+ # 详细输出
# 需要自定义日志目录时，--log-base 是 colcon 全局参数，必须放在 build 前面
colcon --log-base /tmp/my_ws_log build --packages-select my_pkg
```

清理构建产物前先确认当前目录确实是目标工作空间，再删除明确的相对路径；不要从未知目录直接执行宽泛的递归删除命令。

构建后必须 source 工作空间：`source install/setup.bash`。环境叠加顺序：先 `/opt/ros/jazzy/setup.bash`，再 `~/ros2_ws/install/setup.bash`。

讲次 7：工作空间（Workspace）创建与管理

知识点

```
# mkdir -p → 创建目录，-p 表示父目录不存在就一并创建（一次性建好 ros2_ws 和子目录 src）
# && cd ~/ros2_ws → 创建成功后进入工作空间根目录
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws
```

创建新包：

```
# ros2 pkg create → 创建一个新的 ROS 2 功能包（自动生成标准目录和配置文件）
# --build-type ament_cmake → 指定为 C++ 包（用 CMake 构建）
# my_cpp_pkg → 包名，可自定义
# --dependencies rclcpp std_msgs → 声明依赖：rclcpp 是 C++ 客户端库，std_msgs 是标准消息类型
ros2 pkg create --build-type ament_cmake my_cpp_pkg --dependencies rclcpp std_msgs

# --build-type ament_python → 指定为 Python 包（用 setuptools 构建）
# --dependencies rclpy std_msgs → rclpy 是 Python 客户端库，std_msgs 是标准消息类型
ros2 pkg create --build-type ament_python my_py_pkg --dependencies rclpy std_msgs
```

工作空间采用 overlay 模式：/opt/ros/jazzy（underlay）→ ~/ros2_ws（overlay）。src/ 纳入 Git，build/install/log/ 不纳入。依赖安装：rosdep install --from-paths src --ignore-src -r -y。

讲次 8: 【LAB】创建并激活你的第一个 ROS 2 工作空间

知识点

本实验将创建第一个 ROS 2 工作空间，编写一个最小节点，构建并运行。

实验步骤

Step 1: 创建工作空间

```
mkdir -p ~/ros2_ws/src # 创建工作空间，src 目录用于存放所有功能包源码
cd ~/ros2_ws           # 进入工作空间根目录（后面 colcon build 必须在这里执行）
```

Step 2: 创建 Python 包

```
cd ~/ros2_ws/src # 进入 src 目录，新建的包会放在这里
# 创建名为 hello_ros2 的 Python 包，依赖 rclpy（Python 客户端库）和 std_msgs（标准消息）
ros2 pkg create --build-type ament_python hello_ros2 --dependencies rclpy std_msgs
```

Step 3: 编写最小节点

创建 `~/ros2_ws/src/hello_ros2/hello_ros2/talker_node.py` :

```
#!/usr/bin/env python3
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from std_msgs.msg import String

def build_greeting_message() -> String:
    msg = String()
    msg.data = "Hello ROS 2!"
    return msg

class TalkerNode(Node):
    def __init__(self):
        super().__init__("talker")
        self.publisher_ = self.create_publisher(String, "chatter", 10)
        self.timer_ = self.create_timer(1.0, self.timer_callback)
        self.get_logger().info("Talker node started!")

    def timer_callback(self):
        msg = build_greeting_message()
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    node = TalkerNode()
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()
```

参考答案（完整可 build 的包）：仓库 `stage1_code/hello_ros2/`，可复制到 `~/ros2_ws/src/` 后构建。

Step 4: 配置入口点

编辑 `setup.py`，在 `entry_points` 的 `console_scripts` 中添加：

```
entry_points={
    'console_scripts': [
        'talker = hello_ros2.talker_node:main',
    ],
},
```

Step 5: 构建工作空间

```
cd ~/ros2_ws # 回到工作空间根目录
source /opt/ros/jazzy/setup.bash # 先加载 ROS 2 系统环境, colcon 才能找到依赖
# colcon build → 编译工作空间里的包
# --symlink-install → 用软链接安装, Python 改代码后无需重新 build (只需重启节点)
# --packages-select hello_ros2 → 只构建指定的包, 加快速度
colcon build --symlink-install --packages-select hello_ros2

# 参考包自带最小测试: 先证明消息内容正确, 再进入运行验证
colcon test --packages-select hello_ros2 --event-handlers console_cohesion+
colcon test-result --verbose
```

推荐始终使用 `--symlink-install` : Python 代码修改后无需重新 build (只需重启节点), 大幅加速开发循环。Stage 2 将详细讲解此机制。

Step 6: 激活并运行

```
source ~/ros2_ws/install/setup.bash # 加载刚构建好的工作空间, ros2 才能找到 hello_ros2
# ros2 run <包名> <可执行名> → 运行包里的节点: talker 是 setup.py 里配置的入口点名称
ros2 run hello_ros2 talker
```

如果报错: `Package 'hello_ros2' not found` 可以用以下命令查看相应包在当前工作空间是否已经生效

```
ros2 pkg list | grep hello_ros2
```

如果找不到一般都是没有执行:

```
source install/setup.bash
```

预期输出 (每秒一行) :

```
[INFO] [talker]: Talker node started!
[INFO] [talker]: Publishing: "Hello ROS 2!"
[INFO] [talker]: Publishing: "Hello ROS 2!"
...
```

Step 7: 验证话题 (新开终端)

```
source /opt/ros/jazzy/setup.bash # 每开一个新终端都要重新加载 ROS 2 系统环境
source ~/ros2_ws/install/setup.bash # 再加载工作空间环境

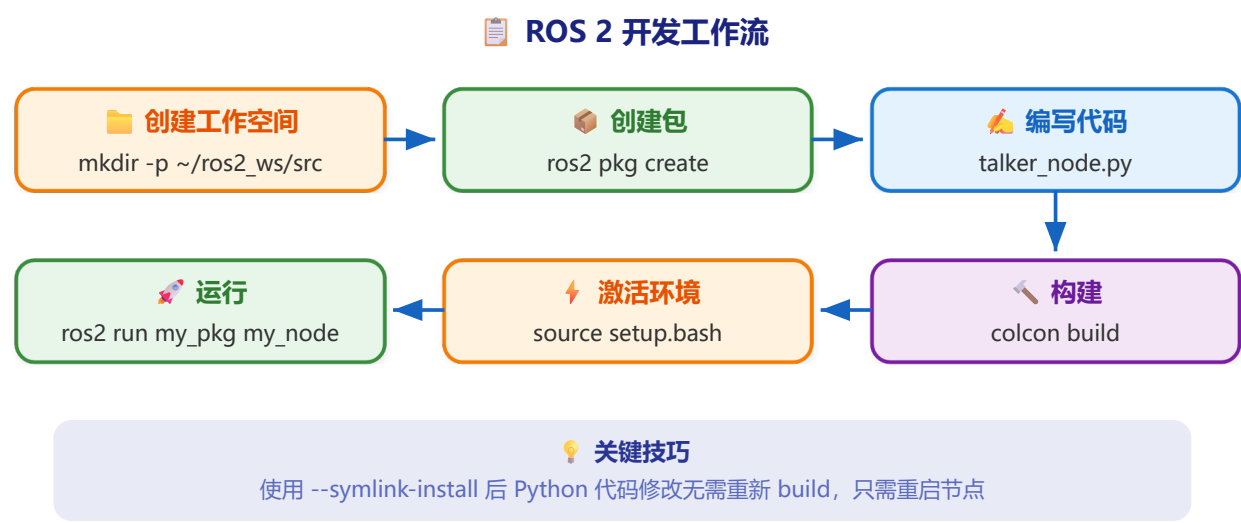
ros2 topic list # 列出当前所有活跃话题
# 预期: /chatter, /parameter_events, /rosout

ros2 topic echo /chatter # 实时打印 /chatter 话题收到的消息内容
# 预期: data: 'Hello ROS 2!'
```

常见错误与解决方案

错误信息	原因	解决方案
Package 'hello_ros2' not found	未 source 工作空间	<code>source ~/ros2_ws/install/setup.bash</code>
No executable found	setup.py 入口点未配置	检查 entry_points 配置后重新 colcon build
colcon: command not found	未安装 colcon	<code>sudo apt install python3-colcon-common-extensions</code>
修改代码后运行旧版本	未重新构建，或修改了入口点/依赖等安装元数据	只改已安装 Python 模块时重启节点即可；修改 setup.py 、资源文件或依赖后仍需重新构建
CMake Error (C++ 包)	缺少依赖	<code>rosdep install --from-paths src --ignore-src -r -y</code>

关键点



图：ROS 2 开发六步 workflow

实验通过标准： 新终端按顺序 source 系统与工作空间环境后，`ros2 topic echo /chatter` 能持续收到 `Hello ROS 2!`。只看到节点进程没有退出，不能证明消息链路正确。

本节总结

关键命令速查表

下表用于安装完成后的日常查阅；APT 源配置、版本检查和首次安装仍以讲次 3 的完整流程为准。

命令	说明
<code>sudo apt install ros-jazzy-desktop</code>	安装 ROS 2 Jazzy 桌面版
<code>source /opt/ros/jazzy/setup.bash</code>	加载 ROS 2 系统环境
<code>sudo rosdep init</code> （仅首次）后执行 <code>rosdep update</code>	初始化并更新依赖索引
<code>mkdir -p ~/ros2_ws/src</code>	创建工作空间
<code>ros2 pkg create --build-type ament_cmake PKG</code>	创建 C++ 包
<code>ros2 pkg create --build-type ament_python PKG</code>	创建 Python 包
<code>colcon build</code>	构建工作空间
<code>source install/setup.bash</code>	加载工作空间环境
<code>ros2 run PKG NODE</code>	运行节点
<code>ros2 topic list</code>	查看活跃话题
<code>ros2 doctor</code>	系统环境诊断
<code>rosdep install --from-paths src --ignore-src -r -y</code>	安装包依赖

下一步

下一节 **1.4 认识讲师与课程路线图** 中，我们将了解课程的整体架构、八大学习阶段、五大实战项目，以及如何高效学习本课程。